

Backend Engineer — Teaching Assignment

Audience: working engineers, 0–3 yrs experience, comfortable in Java, have shipped something small, never taught this properly.

Duration: 120 minutes

00:00–10:00 — Why Do We Need Authentication and Authorisation?

Okay in the opening, I will start with a problem.

[00:00–02:00] : I will say before we start talking about the session, I want to ask you something.

Imagine that we've built a simple e-commerce application.

We have **customers** who can browse products and place orders.
We also have **administrators** who can manage products and users.

Our backend exposes APIs such as:

- GET /products → (for getting the products)
- POST /orders → (to send the order)
- GET /orders → (for getting the orders)
- POST /products → (to send the product)
- DELETE /users/{id} → (to delete the user with a id)

Now imagine that I open Postman and send this request:

- GET /orders

The request reaches our backend.

Here's my first question to you:

→ How does our backend know who is making this request?

[ASK THE CLASS]

"How does the backend know which user is making this API request?"

[EXPECTED ANSWERS]

1. The user
2. User ID
3. Password
4. Owner

[COMMON WRONG ANSWER]: The IP address of the user.

[RESPOND]

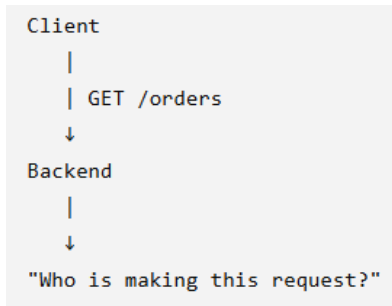
An IP address tells us something about the network connection, but it doesn't reliably tell us which user is making the request.

For example, multiple users can be behind the same network. So an IP address isn't a reliable way of establishing a user's identity.

What we actually need is some mechanism that allows the backend to establish the identity of the person making the request.

[BOARD]

I will write:



Then write:

We need to establish the identity(like who can access it).

[I WILL SAY]

Now let's make the problem a little more interesting.

Suppose, I tell you that the person making this request is me.

Does that automatically mean I'm allowed to delete another user?

[ASK THE CLASS]

If I know who the user is, does that mean I should allow them to perform every operation?

[EXPECTED ANSWER]

No.

[I WILL SAY]

Exactly.

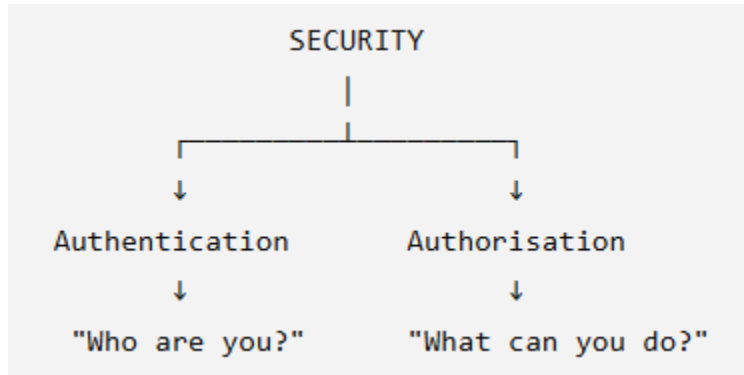
Knowing **who someone is** and knowing **what that person is allowed to do** are two different problems.

And these two problems are going to form the foundation of everything we're going to discuss today.

[02:00–05:00] — Authentication vs Authorization

[BOARD]

Write:



[SAY]

"The first question is:

Who are you?

That's **authentication**.

The second question is:

What are you allowed to do?

That's **authorisation**.

Let's make this concrete.

Suppose our application has two types of users:

- USER
- ADMIN

A normal user might be allowed to:

- GET /products
- POST /orders
- GET /orders

But maybe only an administrator can:

- DELETE /users/{id}
- POST /products
- DELETE /products/{id}

So imagine that Shubhi successfully logs into our application.

At this point, we have authenticated her.

But we still need to ask:

What is Shubhi allowed to do?"

[SAY]

"Right.

She is authenticated, but she may not be authorised to perform that operation.

So I want you to remember these two questions throughout today's session:

Authentication

→ Who are you?

Authorization

→ What are you allowed to do?

When these conditions fail, then you will see some error, it some how look like this, with status code:

401 Unauthorized and 403 Forbidden

AUTHENTICATION → "who are you?" → fails as 401

AUTHORIZATION → "are you allowed to?" → fails as 403

[05:00–07:30] — Introducing the Login Problem

[SAY]

"Now let's solve the first problem.

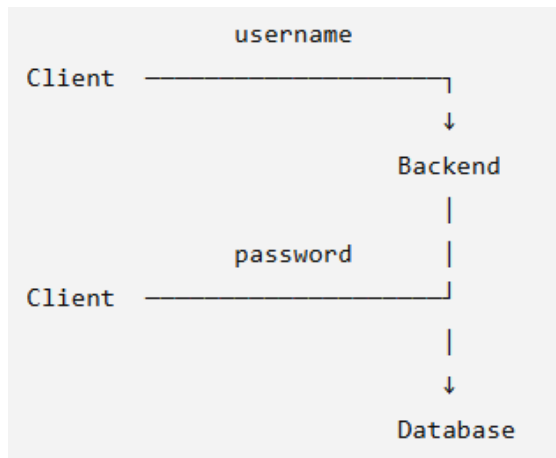
How do we establish someone's identity?

The most familiar approach is a login.

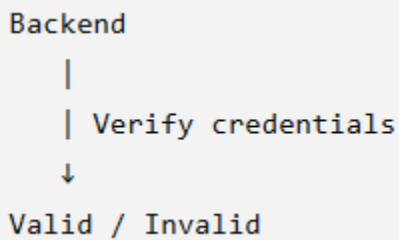
A user provides credentials, usually a username or email and a password."

[BOARD]

Draw:



Then:



[SAY]

"For example, our client sends:

- POST /login

with something like:

```
{  
  "email": "shubhi@example.com",  
  "password": "*****"  
}
```

The backend receives these credentials and checks whether they are valid.

If they're valid, we can say:

We have authenticated the user."

[ASK THE CLASS]

"Should we store the user's actual password directly in our database?"

[EXPECTED ANSWER]

"No."

[FOLLOW-UP]

"Why not?"

[EXPECTED ANSWERS]

- "It's insecure."
- "If the database is compromised, passwords are exposed."

[SAY]

"Correct."

We don't want our database to contain users' original passwords.

Instead, passwords are normally stored using a password-hashing mechanism, and during login the submitted password is verified against the stored password hash.

[07:30–10:00] — The Problem After Login

[SAY]

"Let's say the login was successful."

The backend has verified that this is Shubhi. Great.

But our application doesn't have just one API call.

After logging in, Shubhi might make hundreds of requests."

[BOARD]

Write:

1. POST /login
2. GET /products
3. GET /orders
4. POST /orders

5. GET /profile

[SAY]

"Here's the interesting question.

The login request contained the username and password.

But when Shubhi sends:

GET /orders

should she send her password again?"

[ASK THE CLASS]

"Should the client send the username and password with every request?"

[EXPECTED ANSWER]

"No."

[SAY]

"Right.

Apart from being inconvenient, repeatedly sending credentials is not the model we want to build our entire application around.

So now we have a new problem.

We successfully authenticated the user during login.

"How do we communicate that authenticated identity to the backend on subsequent requests?"

[BOARD]

Write:

```
Login
↓
User is authenticated
↓
How does the next request know that?
↓
????????
```

[PAUSE]

"Take a few seconds and think about that."

What could the client send with the next request so that the server can recognise the authenticated user?"

[EXPECTED ANSWERS]

Students may say:

- Session
- Token
- JWT
- Cookie

[SAY]

"Exactly."

And this is the point where we're ready to introduce the next part of our story: **how we maintain the user's authenticated state across requests.**

We'll first look at the traditional session-based approach.

Then we'll see why token-based authentication became important, and eventually we'll arrive at **JWT**.

So instead of starting with:

'JWT has three parts...'

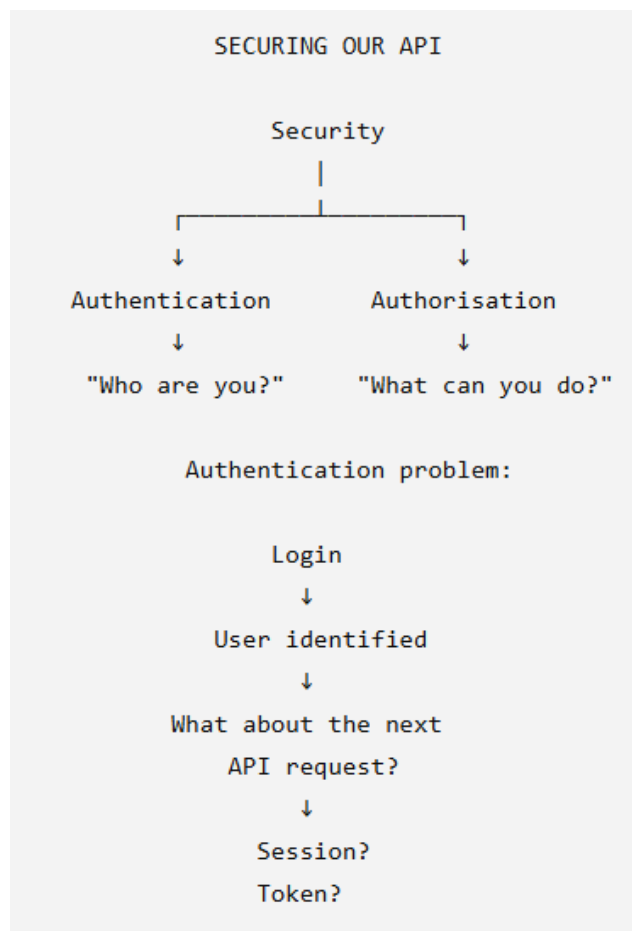
we've now reached JWT because we actually have a problem that needs solving."

[10:00 CHECKPOINT]

[BOARD — FINAL STATE]

[SAY]

Till now, we have studied this far:



[SAY]

"Now that we understand the problem, let's look at the first way a server can remember that a user has already authenticated: **sessions**.

And once we understand sessions, the motivation for tokens — and eventually JWT — will become much clearer."

End of 00:00–10:00 section.

[10:00–12:00] — What Happens After Login?

[SAY]

"Let's continue from where we stopped.

We have a user who has successfully logged in.

The server has verified the user's credentials, so at this point we know: This user is authenticated.

But remember our original problem.

The user is going to make many more requests.

For example:

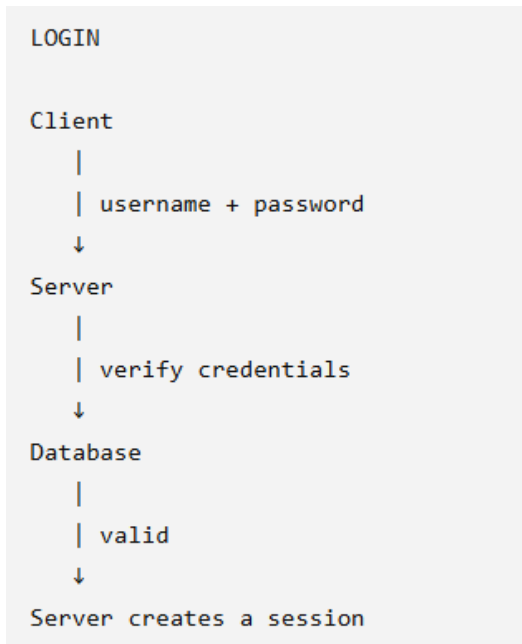
- GET /products
- GET /orders
- POST /orders
- GET /profile

The server needs some way to recognise that these requests belong to the user who already logged in.

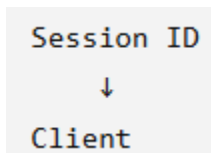
One traditional way of solving this problem is **session-based authentication**."

[BOARD]

Write:



Then:



[SAY]

"The important thing here is that the server creates a **session** after successful authentication.

Think of the session as a record maintained by the server that says:

'This particular session belongs to this authenticated user.'

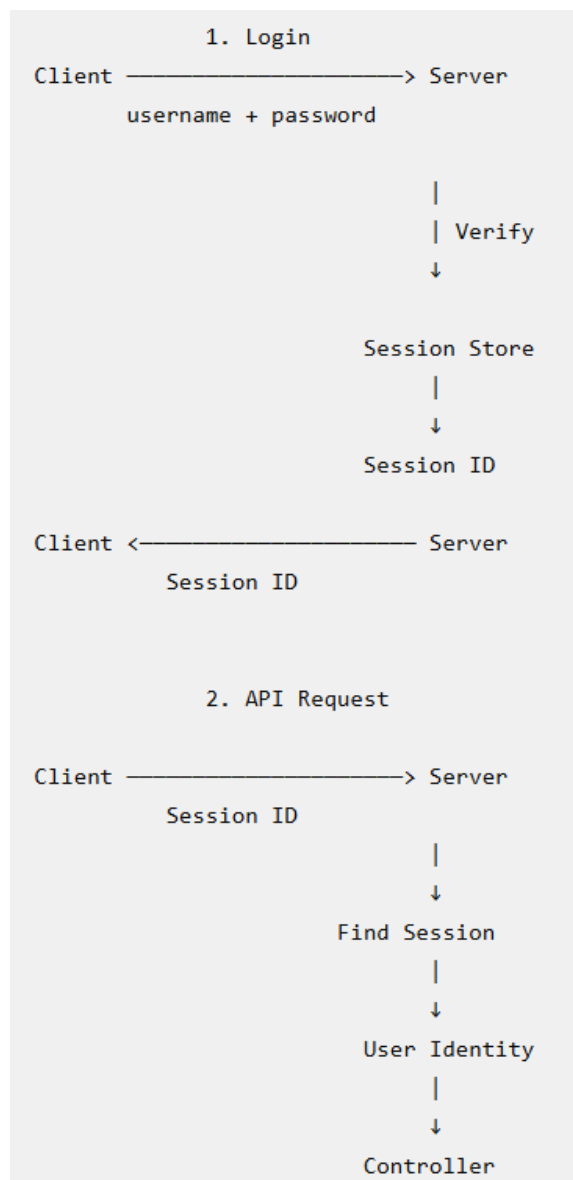
The server then gives the client a **session identifier**.

The client can send that identifier with subsequent requests."

[12:00–15:00] — Understanding the Session Flow

[BOARD]

Draw the complete flow:



[SAY]

"Let's walk through this carefully.

First, the client sends the username and password.

The server verifies the credentials.

If they're valid, the server creates a session.

For example, conceptually, the server might have something like:

Session ID: abc123

User: Shubhi

Authenticated: true

The client receives abc123.

Now, when the client sends another request, it sends that session identifier.

The server looks up the session and says:

'Okay, session abc123 belongs to Shubhi, and Shubhi has already been authenticated.'

Now the server knows who is making the request."

[ASK THE CLASS]

"So where is the actual authenticated state stored in this model — on the client or on the server?"

[EXPECTED ANSWER]

"On the server."

[SAY]

"Correct.

That's an important characteristic of this approach.

The server maintains the session state."

[15:00–17:00] — A Simple Analogy

[SAY]

"Here's a simple analogy.

Imagine you go to an office building.

At the entrance, you show your identity document to the security desk.

The security guard verifies you.

Instead of showing your identity document at every door inside the building, you're given a temporary access badge.

Now when you reach another door, you show the badge.

The door doesn't need to ask you for your identity document again.

The badge points to the fact that you've already been verified.

A session identifier plays a somewhat similar role in our application."

[ASK THE CLASS]

"Does the session ID itself need to contain all the user's information?"

[EXPECTED ANSWER]

"No."

[SAY]

"Exactly.

The session ID can simply identify the session. The server can use it to retrieve the associated user information from its session store."

[17:00–20:00] — Where Does the Session Live?

[SAY]

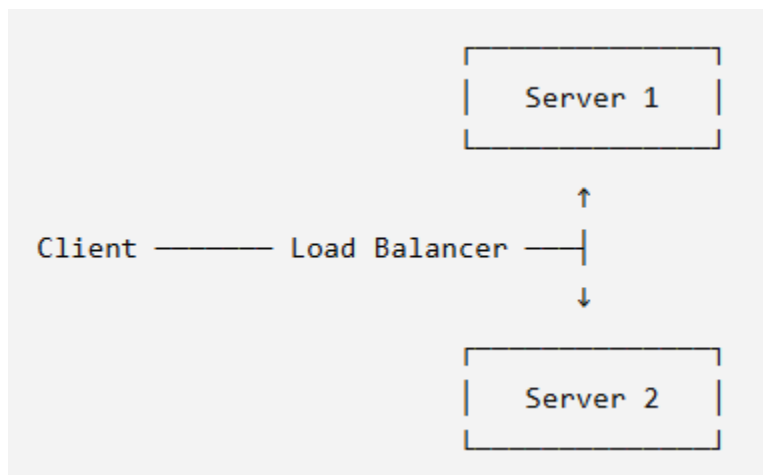
"Now let's make our application slightly more realistic.

Imagine our e-commerce application becomes popular.

Instead of having one backend server, we now have multiple instances."

[BOARD]

Draw:



[SAY]

"Suppose the user logs in and their session is created on Server 1.

Then the next request happens to reach Server 2.

What happens?"

[ASK THE CLASS]

"Can Server 2 find the session if the session only exists in Server 1's memory?"

[EXPECTED ANSWER]

"No."

[SAY]

"Right."

And this is one of the challenges that comes with server-side sessions in a distributed application.

We now need to think about how multiple application instances can share or access session state."

[BOARD]

Add:



[SAY]

"We could introduce a shared session store so that both servers can access the same session information."

But notice what has happened.

Our authentication state is now something the backend infrastructure needs to maintain and share."

[20:00–22:30] — Introducing the Token-Based Idea

[SAY]

"Now let's go back to our original problem.

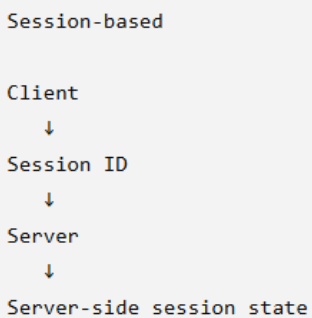
We want the server to know: Who is making this request?

With sessions, the server maintains that state. But what if instead we could give the client a token after login, and the client could present that token on future requests?

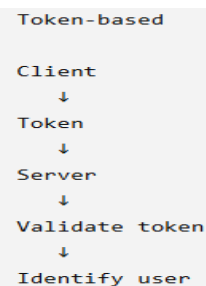
The server could validate that token and use the information associated with it to establish the user's identity."

[BOARD]

Write:



Then:



[SAY]

"This is the basic idea behind token-based authentication.

And this is where we are getting closer to JWT.

But before we jump directly into JWT, I want to make one distinction.

Token-based authentication and JWT are not synonyms.

A token is a general concept.

JWT is a particular **format for representing information in a token.**

We'll come back to that distinction shortly."

[22:30–24:00] — A Common Misconception

[ASK THE CLASS]

"If we're using a token, does that automatically mean we're using JWT?"

[EXPECTED ANSWER]

"No."

[SAY]

"Correct.

JWT is one possible token format.

So don't create this mental model: → Token = JWT

Instead:

Token

↓

One possible format



JWT

We'll understand exactly what JWT gives us in the next section."

[24:00–25:00] — Transition to JWT

[SAY]

"Let's summarize what we've discovered.

We started with a simple problem: The user logs in once, but then makes many API requests.

We looked at sessions, where:

Server



maintains session state

Then we asked: Can we use a token that the client sends with subsequent requests?

That gives us:

```
Login
↓
Token
↓
Client stores token
↓
Client sends token with request
↓
Server validates token
↓
Server identifies user
```

Now we have the problem that we wanted to solve.

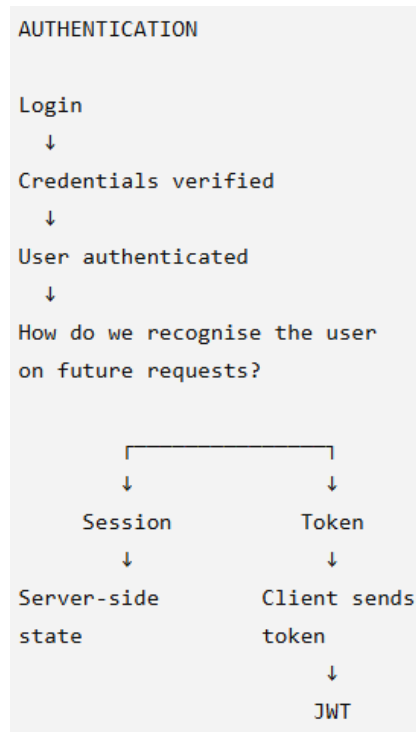
So the next question is:

What exactly should this token look like, and how can the server trust it?

That's where **JWT — JSON Web Token** — comes into our story."

[25:00 CHECKPOINT]

[BOARD — KEEP THIS ON SCREEN]



[QUICK CHECK]

Ask the class:

"What is the main difference in where the authentication state is maintained between a traditional session approach and the token-based approach we've just introduced?"

[EXPECTED ANSWER]

"With sessions, the server maintains session state. With token-based authentication, the client presents a token with subsequent requests, which the server validates."

[TRANSITION]

"Good. Now let's take that token and understand what's actually inside it.

End of 10:00–25:00 section.

25:00–45:00 — JWT: Understanding the Token From First Principles

[25:00–27:00] — What Is a JWT?

[SAY]

"Now let's answer the question we ended the previous section with.

What exactly is this token?

You've probably heard the term **JWT**, or JSON Web Token.

But I don't want us to start by memorising the expansion of JWT.

Let's start with what we need the token to do.

Our client needs to make a request like:

GET /orders
Authorization: Bearer <token>

When that request reaches our backend, the backend needs to be able to answer questions such as:

- Who does this token represent?
- Is this token still valid?
- Has somebody modified the information inside it?

So our token needs to carry some information and, importantly, provide a way for the server to verify that the token hasn't been tampered with."

When that request reaches our backend, the backend needs to be able to answer questions such as:

- Who does this token represent?
- Is this token still valid?
- Has somebody modified the information inside it?

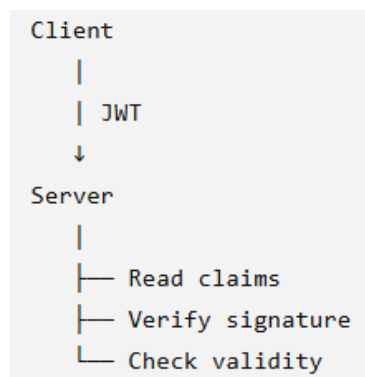
So our token needs to carry some information and, importantly, provide a way for the server to verify that the token hasn't been tampered with."

[BOARD]

Write:

JWT :A compact, URL-safe way of representing claims between parties.

Then:



[SAY]

"JWT is a standard token format.

A JWT can contain **claims** — pieces of information about the subject or the token itself.

For example, conceptually, we might have claims such as:

```
</> JSON

{
  "sub": "12345",
  "role": "USER",
  "exp": 1780000000
}
```

Here we might say:

- **sub** identifies the subject
- **role** tells us the user's role
- **exp** represents the expiration time

These are examples of claims.

But now comes an important question:

If the client can see the token, what's stopping the client from changing **role: USER** to **role: ADMIN**?"

[27:00–30:00] — Why Do We Need a Signature?

[ASK THE CLASS]

"Suppose I receive a token containing the role: **USER**. What stops me from changing it to **role: ADMIN**?"

[POSSIBLE ANSWERS]

- "Encryption"
- "The server checks the database"
- "The JWT is protected"
- "The signature"

[SAY]

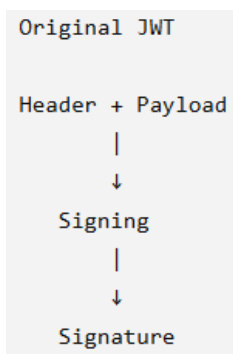
"The key concept here is the **signature**.

A JWT can be digitally signed.

The server can use the signature to verify that the token was produced by a trusted party and that the signed contents haven't been modified."

[BOARD]

Write:



Then:

Header.Payload.Signature

[SAY]

"That gives us the familiar three-part structure of a signed JWT:

Header.Payload.Signature

The three sections are separated by dots."

[30:00–34:00] — Part 1: Header

[SAY]

"Let's take these three parts one at a time.

First, the **header**.

The header contains metadata about the token.

For example:

```
</> JSON  
  
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

alg tells us the algorithm associated with the signature.

typ indicates the token type."

[BOARD]

```
HEADER  
  
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

[ASK THE CLASS]

"Does the header contain the user's password?"

[EXPECTED ANSWER] → "No."

[SAY]

"Correct.

The header is metadata about the token. It isn't where we put sensitive authentication credentials."

[34:00–37:00] — Part 2: Payload and Claims

[SAY]

"The second part is the **payload**.

This is where we put claims.

For example:

```
</> JSON

{
  "sub": "12345",
  "role": "USER",
  "exp": 1780000000
}
```

Here:

sub can represent the subject of the token.

role is a custom claim that our application might use for authorisation.

exp represents the expiration time."

[BOARD]

PAYLOAD

```
{  
  "sub": "12345",  
  "role": "USER",  
  "exp": 1780000000  
}
```

Then:

Claims

↓

Information carried by the JWT

[ASK THE CLASS] → "If I decode the payload, can I read these values?"

[EXPECTED ANSWER] → "Yes."

[SAY] → "Yes."

And this brings us to one of the most important JWT misconceptions."

[37:00–39:00] — Common Misconception: JWT Is Not Automatically Encrypted

[ASK THE CLASS]

"Is the JWT payload encrypted?"

[COMMON WRONG ANSWER]

"Yes, because the contents aren't directly readable when we look at the token."

[SAY]

"Not necessarily.

A typical signed JWT is **encoded, not encrypted**.

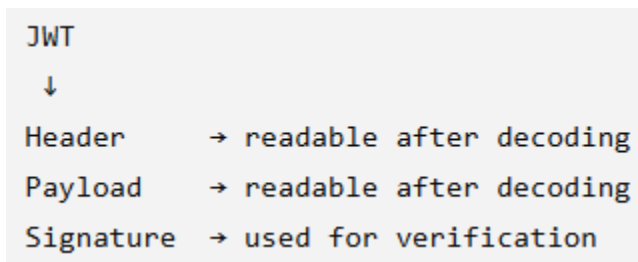
That means someone who has the token can decode its header and payload.

The signature helps us detect tampering; it does not make the payload secret."

[BOARD]

Write: `Encoded ≠ Encrypted`

Then:



[SAY]

"So if you have sensitive information that the client should never be able to see, don't assume that putting it inside a normal signed JWT makes it secret.

A signed JWT gives us integrity and authenticity properties for the signed content; it is not automatically a mechanism for confidentiality."

[39:00–42:00] — Part 3: Signature

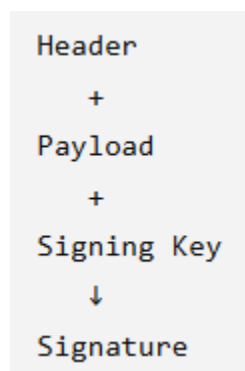
[SAY]

"Now let's look at the third part: the **signature**.

The exact signing process depends on the algorithm, so let's first understand the concept rather than getting lost in cryptographic mathematics.

Conceptually, the signing process takes the header and payload and produces a signature using a secret or private key, depending on the signing algorithm."

[BOARD]



Then:

Header.Payload.Signature

[SAY]

"Now suppose the client changes:



The payload has changed.

The existing signature was generated for the original contents.

When the server verifies the token, the signature check will no longer match the modified contents.

The server can therefore reject the token."

[ASK THE CLASS]

"So does the signature prevent somebody from physically changing the token?"

[EXPECTED ANSWER]

"No."

[SAY]

"Exactly."

This is a subtle but important distinction.

A user can modify the text of a token.

What they **cannot do successfully** is modify the signed content and still produce a valid signature without the appropriate signing key."

[42:00–44:00] — How the Server Validates a JWT

[SAY]

"Let's now put the pieces together."

A client sends:

GET /orders

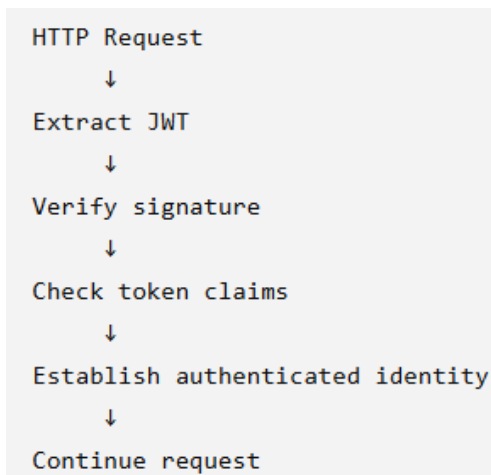
Authorization: Bearer <JWT>

The request reaches our backend. The backend can perform several checks.

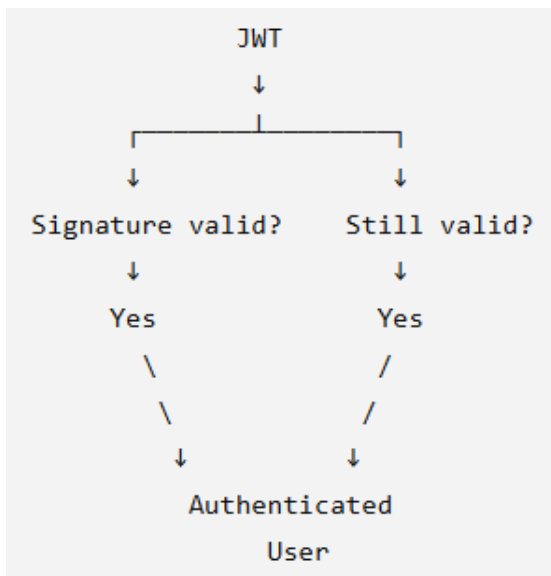
For example, the server may check whether the token is expired.

If the token has an **exp** claim and that expiration time has passed, the token should no longer be accepted as valid."

Conceptually:



[BOARD]



[44:00–45:00] — Quick Checkpoint

[ASK THE CLASS]

"Let's do a quick check before we move on.

A JWT has three parts. What are they?"

[EXPECTED ANSWER]

"Header, Payload and Signature."

[ASK]

"What do we generally find in the payload?"

[EXPECTED ANSWER]

"Claims."

[ASK]

"Is the payload automatically encrypted?"

[EXPECTED ANSWER]

"No. A normal signed JWT is encoded, not encrypted."

[ASK]

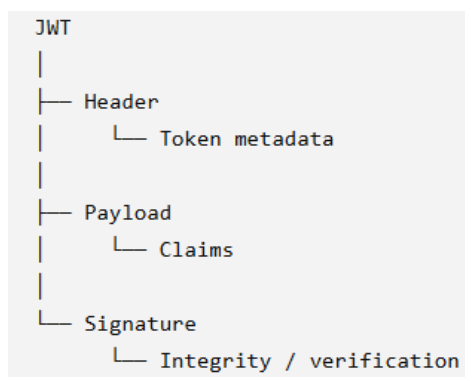
"What does the signature help us with?"

[EXPECTED ANSWER]

"Verifying that the signed contents haven't been tampered with and that the token was signed by a trusted party."

[BOARD — FINAL STATE]

Should be noted properly:



[SAY]

"So now we understand what a JWT is and why its three parts exist.

But we haven't actually used JWT in our Spring Boot application yet.

We've only understood the token itself.

The next question is:

How does Spring Security take this token from an HTTP request, validate it, identify the user, and decide whether that user is allowed to access a particular endpoint?

That's where we're going next — we'll move from **JWT as a concept** to **JWT inside a Spring Boot application.**"

End of 25:00–45:00 Section

45:00–65:00 — JWT in Spring Boot: From Request to Authentication

[45:00–48:00] — Connecting JWT to Spring Security

[SAY]

"So far, we've understood JWT independently.

We know that a JWT can contain claims, and that a signed JWT can be validated by the server. But we're building a **Spring Boot application**.

So the important question is:

When a request containing a JWT reaches our Spring Boot application, who actually looks at that token?

This is where **Spring Security** comes into the picture."

[BOARD]

Write:



[SAY]

"Spring Security provides the security infrastructure that sits in front of our application endpoints. Instead of putting authentication logic inside every controller, we can allow Spring Security to process the request before it reaches the controller."

[ASK THE CLASS]

"Suppose we have 20 protected endpoints. Should we write JWT validation code inside all 20 controllers?"

[EXPECTED ANSWER]

"No."

[SAY]

"Exactly. That would lead to duplicated security logic and would be very difficult to maintain."

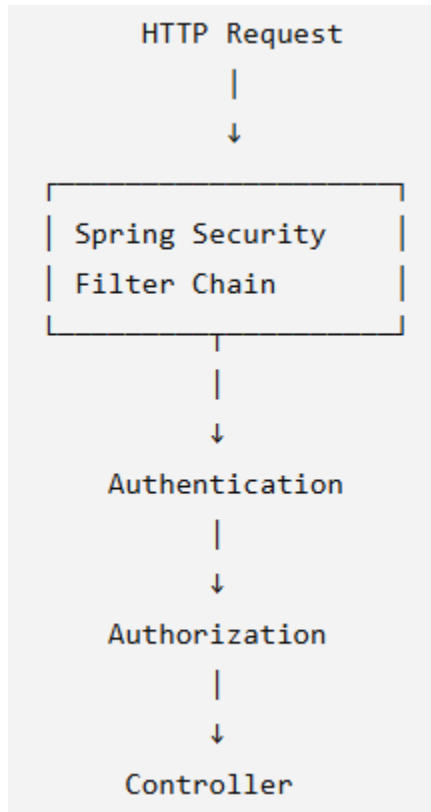
We want security to be handled centrally.

This is one of the reasons the Spring Security filter chain is so important."

[48:00–52:00] — Understanding the Security Filter Chain

[BOARD]

Draw:



[SAY]

"When an HTTP request comes into our application, Spring Security can process that request through a chain of security filters.

Think of these filters as checkpoints before the request reaches our controller.

One filter may be responsible for one type of security-related processing, another may handle something else.

For our JWT-based application, one important responsibility is:

Extract the token from the request, validate it, and establish the authenticated identity."

[ASK THE CLASS]

"Does the JWT validation happen inside our controller?"

[EXPECTED ANSWER] → "No."

[SAY]

"Right. We want the security infrastructure to handle that before our business logic executes."

[52:00–55:00] — What Does the Request Look Like?

[SAY]

"Let's look at an actual HTTP request.

After the user has logged in and received a JWT, the client needs to send that token when accessing a protected endpoint. A common approach is the **Authorization** header."

[BOARD]

Write:

→ GET /orders

→ Authorization: Bearer <JWT>

[SAY]

"There are two important pieces here.

First: Authorization → that's the HTTP header.

Second: Bearer <JWT> → indicates the authentication scheme being used here, followed by the access token.

[ASK THE CLASS]

"Where should the client put the JWT for a typical bearer-token request?"

[EXPECTED ANSWER]

"In the `Authorization` header."

[COMMON WRONG ANSWER]

"In the URL."

[SAY]

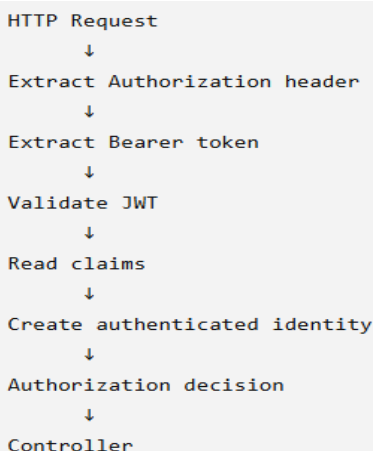
"We generally don't want to put access tokens into URLs. It can end up in places such as browser history, logs, analytics systems, or referrer information. For bearer-token authentication, the standard pattern we're demonstrating is the `Authorization` header."

[55:00–58:00] — What Happens to the JWT?

[SAY]

"Now let's follow the request. The client sends the request, request enters our application.

Conceptually, our security processing does something like this:



Notice something important. The JWT itself doesn't magically give the controller permission to execute.

Spring Security first needs to establish an authenticated security context from the token."

[58:00–61:00] — Authentication Object and Security Context

[SAY]

"Once Spring Security has successfully authenticated the request, the application needs a representation of the authenticated user.

Spring Security uses an **Authentication** object for this purpose.

Conceptually, you can think of it as containing information such as:

- Who is the user?
- What authorities do they have?
- Is the request authenticated?

Spring Security makes this authentication information available through its security context."

[BOARD]



[SAY]

"This separation is useful. Our controller doesn't need to manually parse the JWT every time it wants to know who the current user is.

The security layer establishes the authenticated context, and application code can work with that authenticated identity."

[61:00–63:00] — What If the Token Is Invalid?

[ASK THE CLASS]

"Let's say someone sends us a JWT with an invalid signature.

Should the request reach our controller?"

[EXPECTED ANSWER]

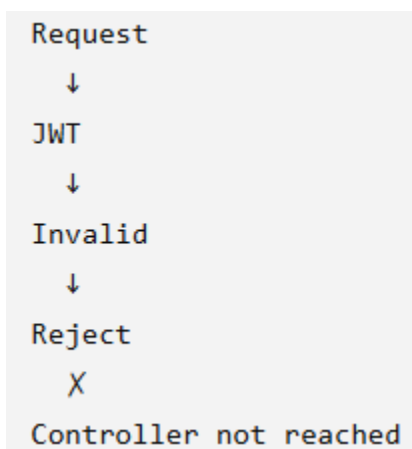
"No."

[SAY]

"Correct.

The security layer should reject the request because we cannot establish a valid authenticated identity from that token."

[BOARD]



[ASK]

"What if there is no token at all and the endpoint requires authentication?"

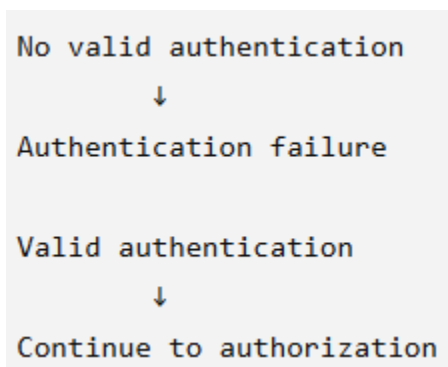
[EXPECTED ANSWER]

"The request should be rejected."

[SAY]

"Exactly."

So we have two different situations:



[63:00–65:00] — Live Coding Setup

[SAY]

"Now I want to stop talking about this abstractly."

Let's build a very small Spring Boot application and secure one endpoint.

We're going to do this incrementally.

I don't want to paste a 100-line security configuration and tell you to memorise it.

Instead, we're going to build it one piece at a time and understand why each piece exists."

[LIVE CODING — STEP 1]

[SCREEN]

Start with a simple controller:

```
</> Java

@RestController
public class OrderController {

    @GetMapping("/orders")
    public String getOrders() {
        return "Orders";
    }
}
```

[SAY]

"At this point, we have a normal endpoint.

If I call: GET /orders

the controller responds. There is currently no authentication requirement."

[LIVE CODING — STEP 2]

[ASK]

"What should happen if /orders is a protected endpoint and I call it without authenticating?"

[EXPECTED ANSWER]

"The request should be rejected."

[SAY]

"Exactly.

So our next task is to tell Spring Security:

`/orders` requires authentication."

[BOARD]

Write:

```
GET /orders
```



```
Is the request authenticated?
```

```
NO → Reject
```

```
YES → Continue
```

[TRANSITION]

[SAY]

"We now have the basic request flow. But there's still a major piece missing.

How does Spring Security actually know that the JWT is valid?

And how do we connect the claims inside that JWT to the authenticated user and their authorities?

That's what we'll build next."

[65:00 CHECKPOINT]

[QUICK QUESTIONS]

Question 1:

"Why do we have a Security Filter Chain?"

Expected:

"To process security concerns before the request reaches the controller."

Question 2:

"Where does the bearer token normally appear?"

Expected:

"In the **Authorization** header."

Question 3:

"If the JWT has an invalid signature, should the controller execute?"

Expected:

"No."

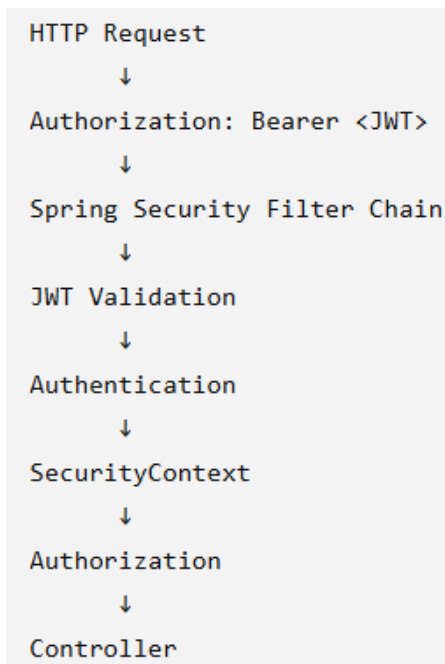
Question 4:

"Is JWT validation something we should duplicate inside every controller?"

Expected:

"No. Security infrastructure should handle it centrally."

[FINAL BOARD]



[SAY]

"We now understand where JWT fits inside Spring Security.

Next, we're going to complete the authentication flow.

We'll look at how the login process generates the JWT, how the token is subsequently validated, and then we'll connect the user's claims and roles to authorisation."

End of 45:00–65:00 section.

65:00–85:00 — Building the JWT Authentication Flow and Introducing Authorisation

[65:00–68:00] — Revisit What We Have Built

[SAY]

"Let's quickly come back to where we are.

We have an e-commerce application.

We have a protected endpoint:

- GET /orders

And we've established that a request should not simply reach the controller just because someone knows the URL. We need to authenticate the user first.

We've also learned that the client can send a JWT using:

- Authorization: Bearer <JWT>

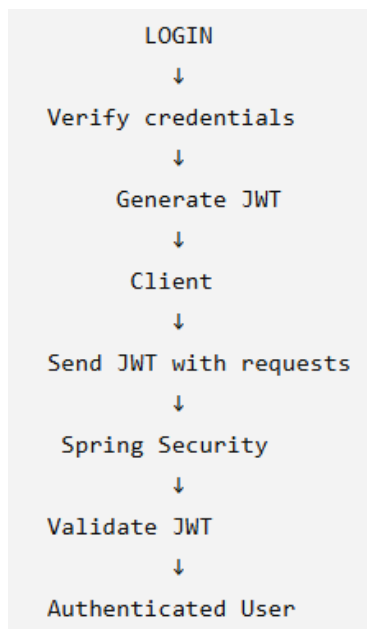
And Spring Security can process that request before it reaches our controller.

But there's still a missing part.

Where did this JWT come from in the first place?

Let's build that flow now."

[BOARD]



[68:00–72:00] — The Login Flow

[SAY]

"Let's imagine the user enters their email and password in our frontend.

The frontend sends with the credentials.

- POST /login

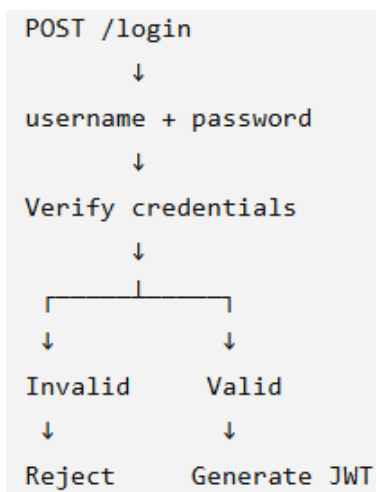
Our backend receives them.

The first responsibility is to verify those credentials.

If the credentials are invalid, authentication fails.

If they're valid, we now know who the user is."

[BOARD]



[ASK THE CLASS]

"At what point should we generate the JWT?"

[EXPECTED ANSWER]

"After successfully authenticating the user."

[SAY]

"Correct. We don't want to issue an authenticated token to somebody whose credentials haven't been verified."

[72:00–75:00] — Generating the JWT

[SAY]

"After successful authentication, our application can create a signed JWT. For example, the token might carry claims representing information our application needs."

[BOARD]

```
</> JSON

{
  "sub": "12345",
  "role": "USER",
  "exp": 1780000000
}
```

[SAY]

"Here we might use:

sub for the subject or user identifier.

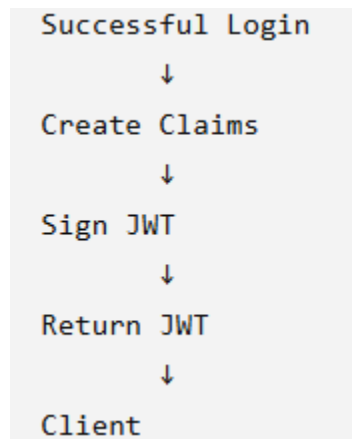
role for an application-specific role claim.

And **exp** for the expiration time.

The token is then signed using the configured signing mechanism.

The client receives the token."

[BOARD]



[ASK THE CLASS]

"Should the client generate the JWT?"

[EXPECTED ANSWER]

"No."

[SAY]

"Right."

In the architecture we're discussing, the trusted backend or authentication component issues the token after authentication. The client receives and presents it; it shouldn't be trusted to simply declare its own identity."

[75:00–78:00] — The Next Request

[SAY]

"Now let's say the frontend wants to retrieve the user's orders."

It sends; Now we're back at the security filter chain."

```
GET /orders
Authorization: Bearer <JWT>
```

[BOARD]



[SAY]

"The important thing is that **login and subsequent authentication are two related but different stages.**

Login establishes the initial authentication and results in a token being issued. Subsequent requests present that token so the server can authenticate the request."

[78:00–80:00] — Authentication Is Not Authorization

[SAY]

"Now we're ready for the second half of our original security problem.

Remember our two questions? Let's bring them back."

[BOARD]

Authentication → Who are you?

Authorization → What can you do?

[SAY]

"Suppose the JWT has been successfully validated.

Spring Security now knows that this request is associated with Shubhi.

Does that mean Shubhi can access every endpoint?"

[ASK THE CLASS]

"Can an authenticated USER call an ADMIN-only endpoint?"

[EXPECTED ANSWER]

"No."

[SAY]

"Exactly.

Authentication answers → Who is this?

Authorization answers → Is this authenticated user allowed to perform this operation?"

[80:00–83:00] — Roles and Authorities

"Let's give our users some permissions.

Suppose we have:

USER, ADMIN

A normal user can:

- GET /products
- GET /orders
- POST /orders

An administrator can additionally:

- POST /products
- DELETE /products/{id}
- DELETE /users/{id}

So after authentication, Spring Security needs some representation of what the user is allowed to do. This is where concepts such as **authorities and roles** become relevant."

[ASK THE CLASS]

"If a user has been authenticated but doesn't have the required authority for an endpoint, should we return an authentication error?"

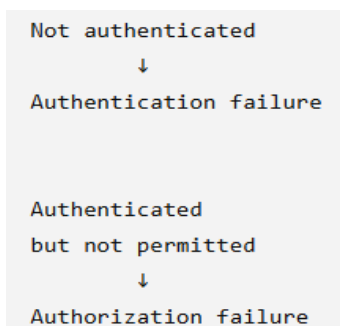
[EXPECTED ANSWER]

"No. The user is authenticated; they simply don't have permission."

[SAY]

"Exactly. This distinction is important. We have two different failure situations."

[BOARD]



[SAY]

"In HTTP APIs, these situations are commonly represented differently — for example, an unauthenticated request may result in **401 Unauthorized**, while an authenticated user who lacks sufficient permission may receive **403 Forbidden**.

The exact response handling depends on the application's security configuration, but the conceptual distinction is what I want you to remember."

[83:00–85:00] — Protecting Endpoints

[SAY]

"Now let's apply this to our Spring Boot application.

We might have an endpoint that should only be available to administrators.

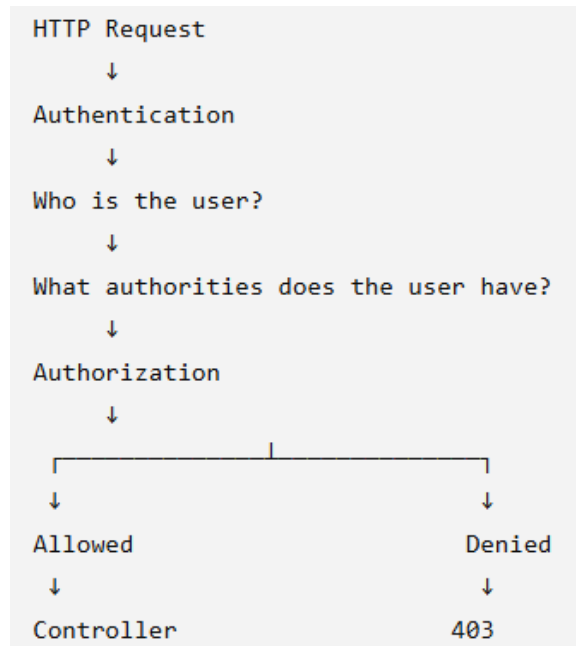
Conceptually, we're expressing a rule like:

```
DELETE /users/{id}
    ↓
Requires ADMIN authority
```

Spring Security can enforce these rules before the controller executes.

So the complete flow becomes:"

[BOARD]



[85:00 CHECKPOINT]

[ASK THE CLASS]

Question 1:

"When do we normally issue the JWT in this flow?"

Expected answer:

"After successfully authenticating the user's credentials."

Question 2:

"Who should be responsible for issuing the authenticated token?"

Expected answer:

"A trusted backend/authentication component, not the client simply declaring its own identity."

Question 3:

"If the JWT is valid, does that automatically mean the user can access every endpoint?"

Expected answer:

"No. Authentication and authorization are separate."

Question 4:

"What's the conceptual difference between 401 and 403?"

Expected answer:

"401 relates to failure to establish authentication; 403 means the request is authenticated but not permitted."

[TRANSITION]

[SAY]

"So far we've built authentication using credentials and JWT, and we've seen how authorization can restrict what an authenticated user is allowed to do.

But our application still has another interesting problem.

What if we don't want our application to directly handle the user's credentials at all?

For example, what if we want the user to click: '**Continue with Google**'

Who authenticates the user then?

And how does our application receive the result?

That's the problem that leads us to **OAuth2**."

End of 65:00–85:00 section.

85:00–105:00 — OAuth2: Why Do We Need It?

[85:00–88:00] — Start With a New Problem

[SAY]

"Let's pause our current implementation for a moment.

Our application currently has its own login system.

A user provides: **Email & Password**

Our application verifies those credentials, authenticates the user, and issues a token. That works.

But now imagine we're building a consumer application and we want to give users another option:

'Continue with Google'

We've all seen this button.

So here's the question: **When the user clicks 'Continue with Google', should our application ask the user for their Google password?"**

[ASK THE CLASS]

"Should our application receive and store the user's Google password?"

[EXPECTED ANSWER]

"No."

[SAY]

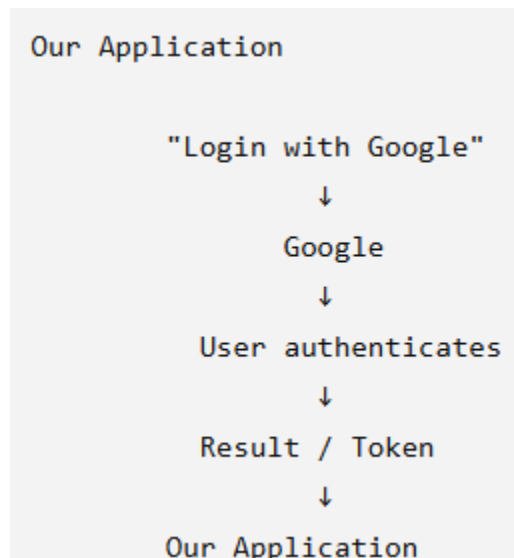
"Exactly.

We don't want our application to know the user's Google password.

Instead, we want Google to handle authentication and then tell our application:

This user has successfully authenticated, and here is the information/token your application is allowed to use."

[BOARD]



[88:00–91:00] — Why OAuth2 Exists

[SAY]

"This is where **OAuth 2.0** comes into our story.

At a high level, OAuth 2.0 gives applications a standard way to obtain **delegated access** to resources without requiring the application to collect the user's credentials for that resource.

Let's use a slightly different example.

Imagine our application wants to access some resource on behalf of a user.

We don't want the user to give our application their Google password.

Instead, the user can authenticate and grant the appropriate access through the provider."

[BOARD]

Write:

Without OAuth-style delegation:

```
User
  ↓
Gives password
  ↓
Our Application
  ↓
Access resource
```

Then:

With delegated authorization:

```
User
  ↓
Authorization Server
  ↓
Grant access
  ↓
Access Token
  ↓
Our Application
```

[SAY]

"The important idea is:

The application gets a token representing granted access instead of collecting the user's password for the external service."

[91:00–94:00] — The Important OAuth2 Actors

[SAY]

"OAuth2 becomes much easier once we understand the actors involved.

Rather than memorising four names, let's identify the responsibilities."

[BOARD]

1. Resource Owner
2. Client
3. Authorization Server
4. Resource Server

[SAY]

"First, the Resource Owner : This is typically the user who owns or controls access to some protected resource.

Second, the Client : This is the application requesting access.

Third, the Authorization Server: This is responsible for authenticating the user and issuing tokens after the appropriate authorization has been granted.

And fourth, the Resource Server: This is the server hosting the protected resource and accepting access tokens."

[ASK THE CLASS]

"In our 'Continue with Google' example, who is the client?"

[EXPECTED ANSWER]

"Our application."

[SAY]

"Correct."

[ASK]

"Who is the authorization server?"

[EXPECTED ANSWER]

"Google's authorization service."

[SAY]

"Right, conceptually.

And if we're requesting access to a protected resource hosted by Google, the corresponding resource server would be the service hosting that resource."

[94:00–98:00] — Walk Through the Authorization Code Flow

[SAY]

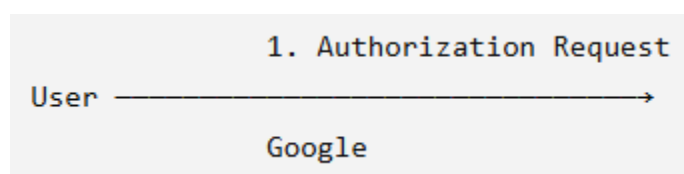
"Let's now walk through a common OAuth2 flow.

We'll use the **Authorization Code flow**, because it illustrates the separation between authorization and obtaining an access token very well.

Imagine the user clicks → Continue with Google

Our application redirects the user to the authorization server."

[BOARD]



[SAY]

"The user interacts with the authorization server.

The user authenticates there and gives the requested consent, if required."

[BOARD]

Add:



[SAY]

"After successful authorization, the authorization server redirects the user back to our application with an **authorization code**.

Now here's an important point.

The authorization code is **not the same thing as the access token**.

Our application can exchange that code with the authorization server for tokens."

[BOARD]



[98:00–101:00] — Access Token

[SAY]

"Now our application has an **access token**."

The access token represents the access that has been granted.

The application can present that access token when requesting protected resources."

[BOARD]



[ASK THE CLASS]

"Is an access token the same thing as a user's password?"

[EXPECTED ANSWER]

"No."

[SAY]

"Correct. The access token is a credential representing authorization to access certain protected resources.

It should be treated as sensitive because possession of a bearer access token can allow someone to use the permissions associated with that token."

[101:00–103:00] — OAuth2 vs JWT

[SAY]

"Now I want to address one of the most common sources of confusion.

We've talked about OAuth2.

We've also talked about JWT.

Are they the same thing?"

[ASK THE CLASS]

"Is OAuth2 a type of JWT?"

[EXPECTED ANSWER]

"No."

[SAY]

"Correct.

Let's put this on the board."

[BOARD]

```
graph TD; A[OAuth 2.0] --> B[Authorization framework/protocol]; C[JWT] --> D[Token format];
```

The diagram consists of two separate vertical flows. The first flow starts with 'OAuth 2.0' at the top, followed by a downward-pointing arrow, and then 'Authorization framework/protocol' at the bottom. The second flow starts with 'JWT' at the top, followed by a downward-pointing arrow, and then 'Token format' at the bottom. The entire diagram is enclosed in a light gray rectangular box.

[SAY]

"OAuth2 defines how authorization can be delegated and how an application can obtain and use access tokens.

JWT, on the other hand, defines a format for representing claims in a compact token.

So these are different concepts.

An OAuth2 access token **can** be a JWT, but it doesn't have to be.

And a JWT can also be used in situations that aren't an OAuth2 flow."

[ASK THE CLASS]

"So if someone says, 'We're using JWT, therefore we're using OAuth2,' is that necessarily true?"

[EXPECTED ANSWER]

"No."

[SAY]

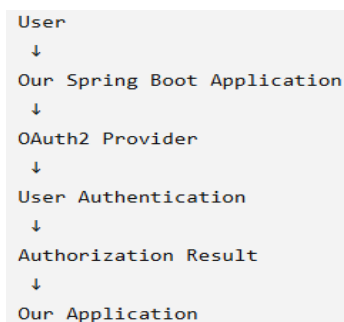
"Exactly."

[103:00–105:00] — Connect OAuth2 Back to Spring Boot

[SAY]

"Now let's bring this back to our Spring Boot application. Our application can act as an OAuth2 client when it wants to delegate authentication or authorization to an external provider.

For example:



Spring Security provides support for implementing these kinds of flows.

But notice how much easier this is to understand now.

We're no longer starting with Spring configuration.

We first understood **why the flow exists**.

We had a problem: We don't want our application to collect another service's credentials.

OAuth2 gives us a standardized way to delegate access.

Then JWT gives us one possible way of representing token claims."

[105:00 CHECKPOINT]

[ASK THE CLASS]

Question 1:

"Why don't we want our application to collect a user's Google password?"

Expected answer:

"Because the external provider should handle authentication, and our application shouldn't need to know the user's password."

Question 2:

"Who is the client in an OAuth2 flow?"

Expected answer:

"The application requesting access."

Question 3:

"What is the role of the authorization server?"

Expected answer:

"It authenticates the user and handles authorization/consent and token issuance."

Question 4:

"Is the authorization code the same as the access token?"

Expected answer:

"No. The authorization code is exchanged for tokens."

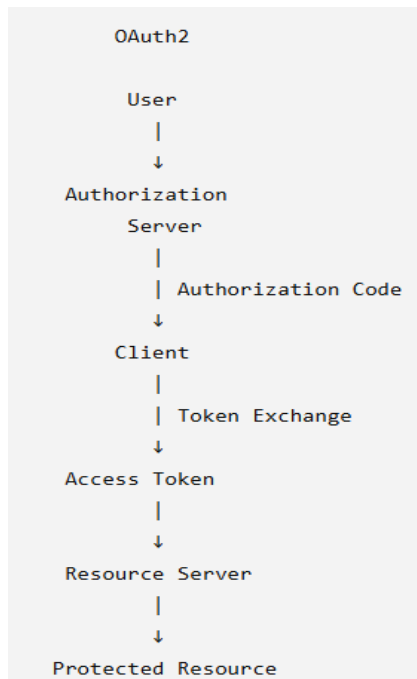
Question 5:

"Is OAuth2 the same thing as JWT?"

Expected answer:

"No. OAuth2 is an authorization framework/protocol, while JWT is a token format."

[BOARD — FINAL STATE]



Underneath:

```
OAuth2 ≠ JWT
```

```
OAuth2 → authorization/delegation flow
```

```
JWT    → token format
```

[TRANSITION TO 105:00–120:00]

[SAY]

"Let's take a step back.

We started this class with a very simple question:

How does our backend know who is making a request?

From that one question, we've built several concepts.

We learned about authentication.

We learned about authorization.

We saw why we might use tokens.

We understood JWT and its structure.

We saw how Spring Security fits into request processing.

And finally, we saw why OAuth2 exists when authentication or authorization needs to involve another trusted provider.

Now I want to put **all of these pieces together**.

We're going to finish by looking at the complete request flow and then test whether we can reason about some real-world security situations."

End of 85:00–105:00 section.

105:00–120:00 — Complete End-to-End Flow, Real-World Scenarios and Final Recap

[105:00–108:00] — Reconstruct the Entire Story

[SAY]

"We've covered a lot of concepts, so I don't want to finish by simply giving you another list of definitions.

Instead, let's reconstruct the entire story from the beginning.

Imagine a user opens our e-commerce application and wants to see their orders.

What happens?"

[BOARD]

Start with only:

User

↓

GET /orders

[ASK THE CLASS]

"What does the backend need to know before it gives the user their orders?"

[EXPECTED ANSWER]

"Who the user is."

[SAY]

"Good. That's our **authentication** problem."

Add:

Authentication



Who is the user?

[SAY]

"Now suppose we know the user is Shubhi.

Do we immediately allow every operation?"

[EXPECTED ANSWER]

"No."

[SAY]

"Right. We also need to determine what Shubhi is allowed to do."

Add:

Authorization



What can the user do?

[108:00–111:00] — Complete JWT Request Flow

[SAY]

"Now let's put JWT into this flow.

The user first logs in."

[BOARD]

1. Login

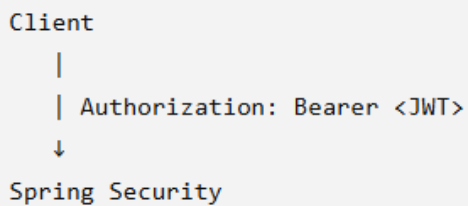


[SAY]

"After successful authentication, the backend issues a signed JWT.

The client then makes a protected request."

2. Protected Request



Then I will:



[SAY]

"So our request doesn't simply go: Request → Controller

Instead, conceptually, we have:



[111:00–114:00] — Real-World Scenario: Invalid Token

[SAY]

"Let's test our understanding with a few situations.

Scenario one.

A client sends:

```
</> http  
  
GET /orders  
Authorization: Bearer <invalid-token>
```

The token's signature doesn't validate.

What should happen?"

[ASK THE CLASS]

"Should the request reach the controller?"

[EXPECTED ANSWER]

"No."

[SAY]

"Correct."

We cannot establish a valid authenticated identity from that token, so the protected request should be rejected."

[BOARD]

```
Invalid JWT
  ↓
Authentication fails
  ↓
Request rejected
  ↓
Controller not executed
```

[114:00–116:00] — Real-World Scenario: Authenticated but Not Allowed

[SAY]

"Scenario two."

The JWT is perfectly valid.

It belongs to a user with the **USER** role.

The user requests: DELETE /users/10

And this endpoint requires an administrator.

What happens?"

[ASK THE CLASS]

"Is authentication successful?"

[EXPECTED ANSWER]

"Yes."

[ASK]

"Is authorization successful?"

[EXPECTED ANSWER]

"No."

[SAY]

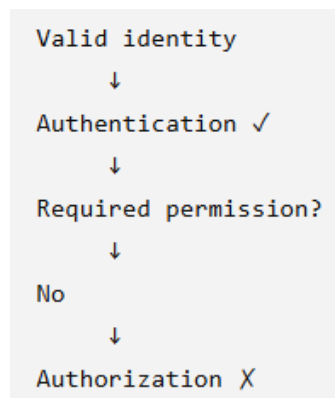
"Exactly.

This is a very important distinction.

The user is **authenticated**.

But the user is **not authorized** for this operation.

Conceptually:



"That's why we shouldn't treat authentication and authorization as the same thing."

[116:00–118:00] — Rapid-Fire Misconception Check

[SAY]

"Before we finish, I'm going to give you a few statements.

Tell me whether they're correct or incorrect."

[QUESTION 1]

"JWT is encrypted, so nobody can read its payload."

[EXPECTED ANSWER]

"Incorrect."

[SAY]

"A typical signed JWT is encoded, not encrypted. Its payload can be decoded. The signature provides verification of the signed content, not confidentiality."

[QUESTION 2]

"If I'm authenticated, I'm automatically authorized to access every endpoint."

[EXPECTED ANSWER]

"Incorrect."

[SAY]

"Authentication establishes identity. Authorization determines whether that identity has permission to perform an operation."

[QUESTION 3]

"OAuth2 and JWT are the same thing."

[EXPECTED ANSWER]

"Incorrect."

[SAY]

"OAuth2 is an authorization framework/protocol, while JWT is a token format. An OAuth2 access token can be a JWT, but OAuth2 does not mean JWT."

[QUESTION 4]

"We should put JWT validation code inside every controller."

[EXPECTED ANSWER]

"Incorrect."

[SAY]

"Spring Security gives us security infrastructure that can process requests before they reach our controllers. We don't want duplicated authentication logic throughout our business code."

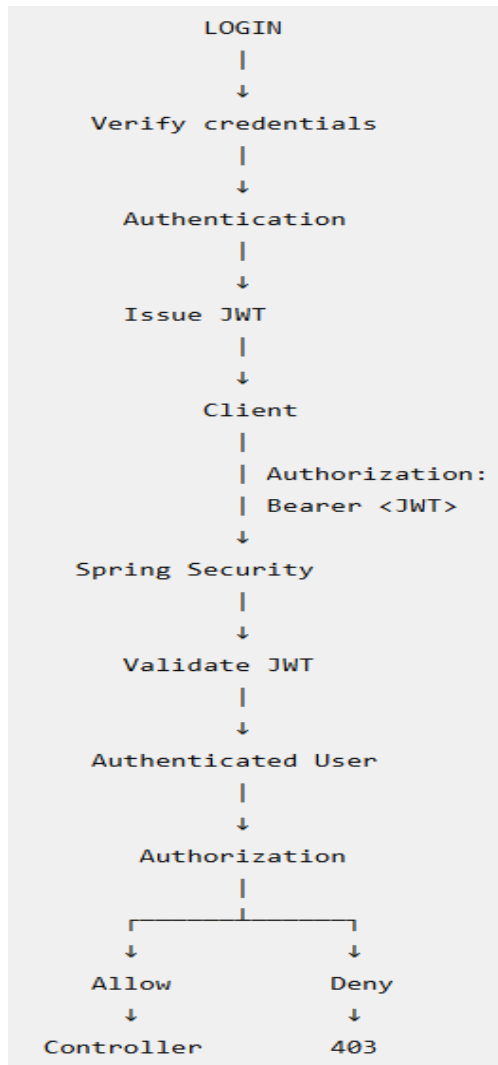
[118:00–120:00] — Final Mental Model

[SAY]

"Let's finish by reducing everything we've learned today to one picture."

[BOARD]

Write the complete flow:



[SAY]

"If you understand this flow, you understand the core story we were trying to build today.

Let's say the four key ideas one last time.

Authentication: Who are you?

Authorization: What are you allowed to do?

JWT: A token format that can carry claims and, when signed, allows the recipient to verify the signed contents.

OAuth2: A standard framework for delegated authorization and obtaining access tokens."

[FINAL ASK]

"Last question.

Imagine tomorrow you join a Spring Boot project and someone tells you:

'We use Spring Security with JWT and OAuth2.'

What are the first questions you would ask?

Don't start with implementation details.

Start with the architecture:

- Who authenticates the user?
- Who issues the token?
- What kind of token are we using?
- How is the token validated?
- What does the application use for authorization?
- Which endpoints require which permissions?
- And if we're using OAuth2, what role does our application play in that flow?"

[CLOSING]

[SAY]

"That's the important shift I want you to take away from this session.

Security isn't about memorising annotations, configuration classes, or token formats.

Start with the questions:

- **Who are you?**
- **What are you allowed to do?**

Once those questions are clear, the tools — Spring Security, JWT, OAuth2 and the configuration around them — become much easier to reason about.

That's where we'll stop."

END.

